1) Word2Vec

Word2Vec is a neural network-based technique used to convert words into continuous vector representations (embeddings). It was developed by Google in 2013 by Tomas Mikolov.

It represents words in a high-dimensional vector space such that semantically similar words are located close to each other.

For example:

- king – man + woman ≈ queen

Key Idea

Words that appear in similar contexts have similar meanings.

Architectures in Word2Vec

Word2Vec has two main models:

1. **CBOW (Continuous Bag of Words)**
2. **Skip-Gram**

Applications

- Semantic similarity
- Text classification
- Machine translation
- Chatbots
- Sentiment analysis

---

2) CBOW (Continuous Bag of Words)

CBOW predicts the **target word** using surrounding context words.

Example:

Sentence: "The cat sits on the mat"

If target word = "sits"
Context words = ["The", "cat", "on", "the"]

CBOW tries to predict "sits" from surrounding words.

Working:

- Input: Context words
- Output: Target word
- Faster to train than Skip-Gram
- Works well with large datasets

Simple CBOW Code (Using Gensim)

```
from gensim.models import Word2Vec

sentences = [
    ["the", "cat", "sits", "on", "the", "mat"],
    ["the", "dog", "runs", "in", "the", "park"]
]

model = Word2Vec(sentences, vector_size=100, window=2, min_count=1, sg=0)

print(model.wv["cat"])
```

sg=0 indicates CBOW.

---

3) GloVe (Global Vectors for Word Representation)

GloVe is an unsupervised learning algorithm developed by Stanford.

Unlike Word2Vec, GloVe uses **global word-word co-occurrence statistics** from a corpus.

Key Idea:

It constructs a co-occurrence matrix and learns word vectors by factorizing this matrix.

Advantages:

- Uses global statistical information
- Captures semantic relationships well
- Efficient for large corpora

Mathematical Concept:

GloVe minimizes a cost function based on the ratio of word co-occurrence probabilities.

---

4) Difference Between Word2Vec and GloVe

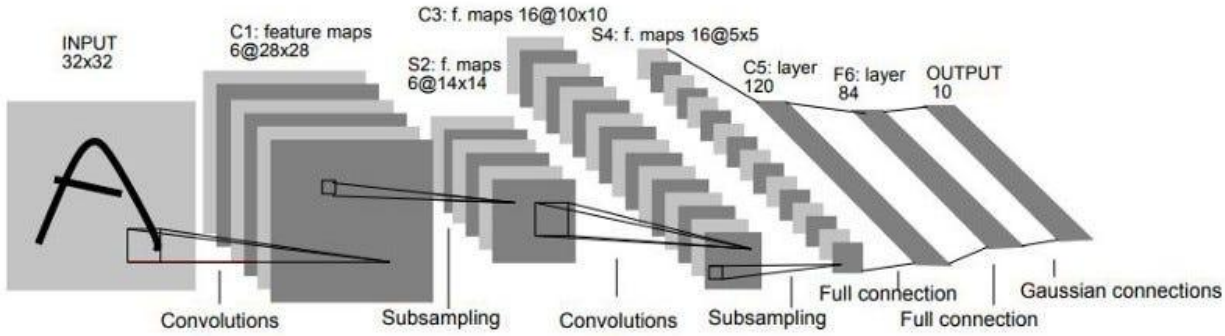
Feature	Word2Vec	GloVe
Approach	Predictive model	Count-based model
Training	Local context window	Global co-occurrence matrix
Method	Neural network	Matrix factorization
Speed	Faster	Slightly slower
Memory	Efficient	Requires large co-occurrence matrix

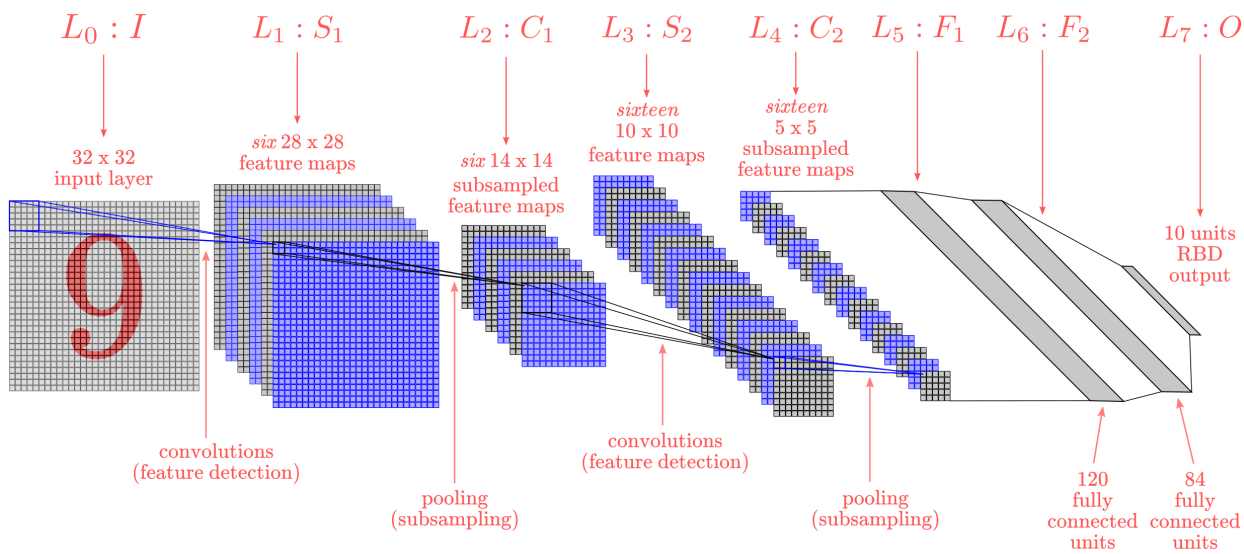
Summary:

- Word2Vec learns embeddings by predicting words.
- GloVe learns embeddings using statistical word co-occurrences.

---

5) What is LeNet? (with Sample Code)





LeNet-5 is one of the earliest Convolutional Neural Networks (CNNs) developed by Yann LeCun in 1998.

It was designed for handwritten digit recognition (MNIST dataset).

Architecture:

1. Convolution layer
2. Average Pooling
3. Convolution
4. Pooling
5. Fully connected layers
6. Output layer

Sample Code (PyTorch)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.pool = nn.AvgPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
```

```
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 4 * 4)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

---

6) What is LlamaIndex? (with Sample Code)

LlamaIndex is a data framework that helps connect Large Language Models (LLMs) with external data sources such as PDFs, databases, APIs, and documents.

It is mainly used for:

- Retrieval-Augmented Generation (RAG)
- Chat with your documents
- Knowledge base integration

Use Cases:

- AI chatbots
- Document Q&A systems
- Enterprise search tools

Sample Code

```
from llama_index import SimpleDirectoryReader, VectorStoreIndex

documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)

query_engine = index.as_query_engine()
response = query_engine.query("What is the main topic?")
print(response)
```

---

7) What are VGG Nets? (with Sample Code)

VGGNet was developed by the Visual Geometry Group (Oxford).

It uses very small (3×3) convolution filters and deep architecture.

Types:

- VGG16
- VGG19

Characteristics:

- Deep network
- Uses 3×3 convolution layers
- Fully connected layers at end

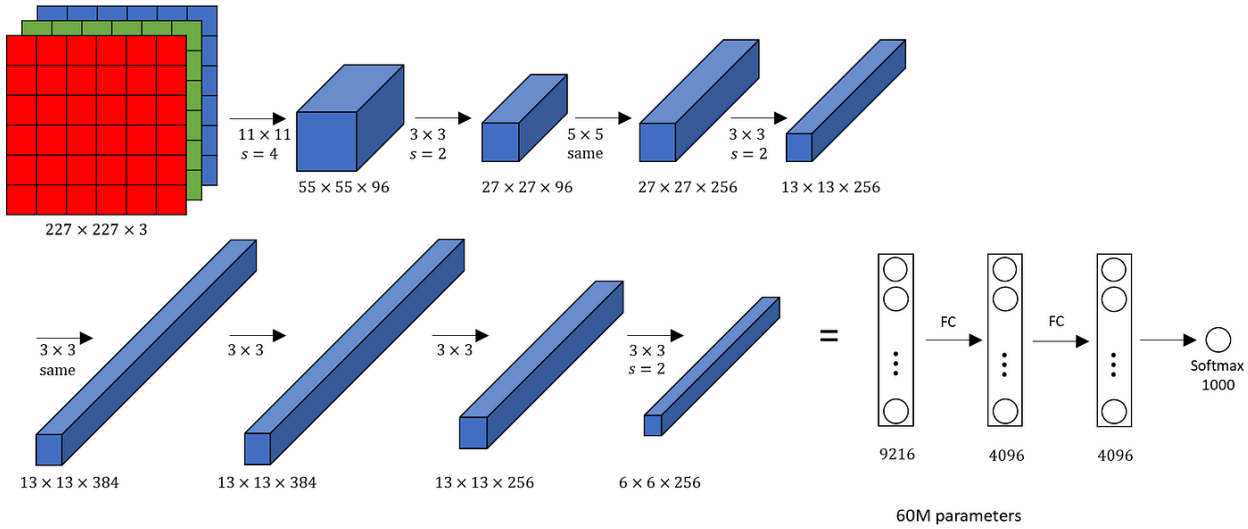
Sample Code (Keras)

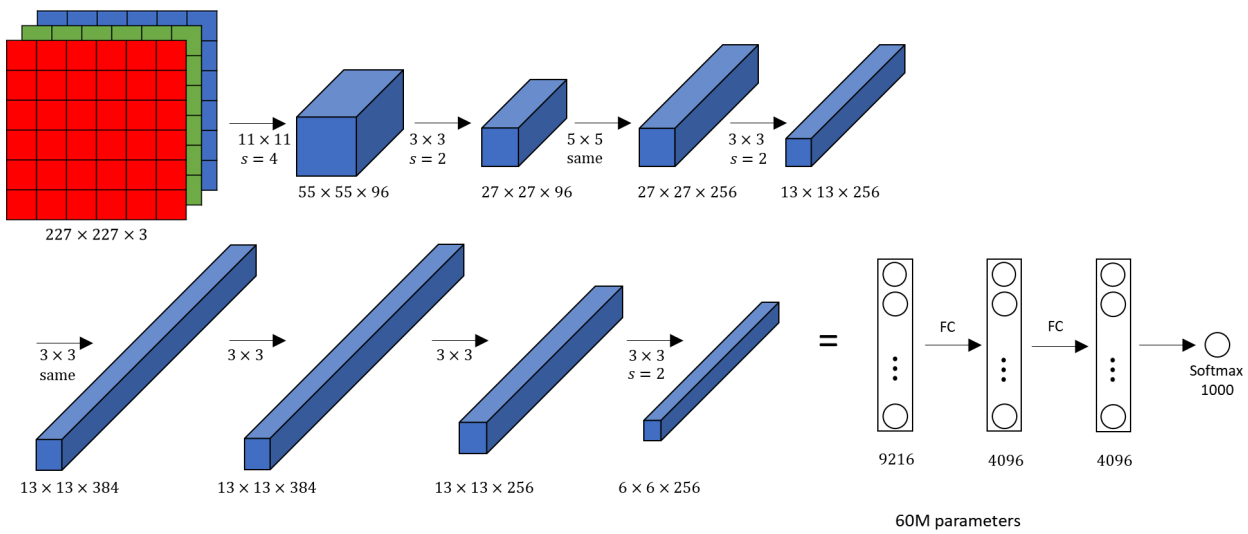
```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model

model = VGG16(weights='imagenet')
model.summary()
```

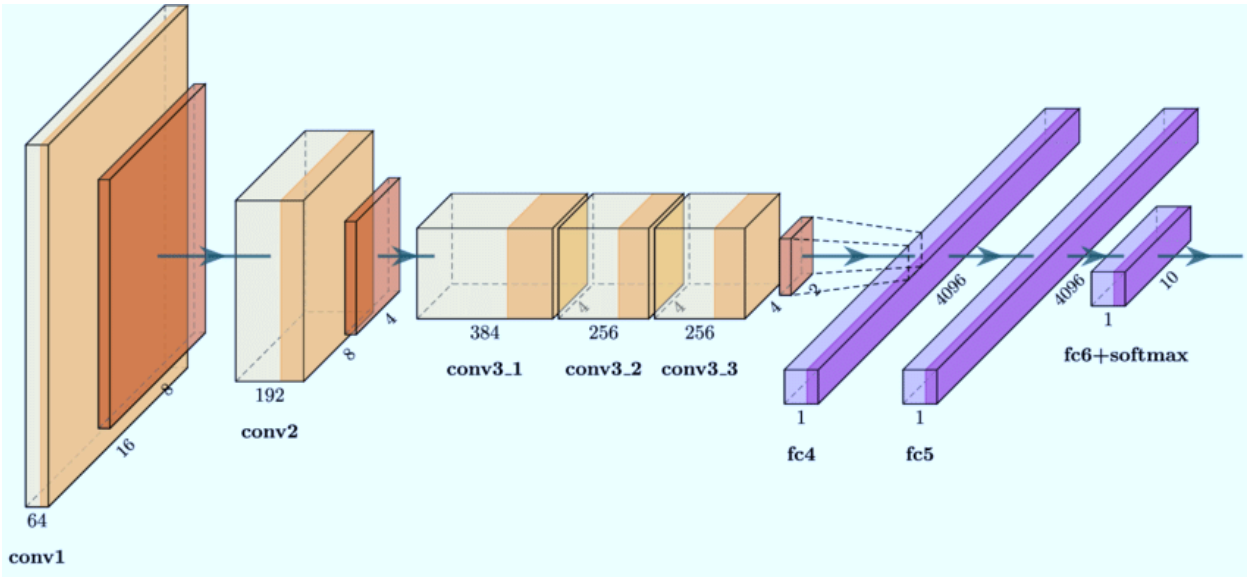
---

8) Explain AlexNet and VGGNet

AlexNet







AlexNet was developed by Alex Krizhevsky in 2012.

It won the ImageNet competition in 2012 and revolutionized deep learning.

Key Features:

- 8 layers
- ReLU activation
- Dropout
- Data augmentation
- GPU training

VGGNet

VGGNet improved over AlexNet by:

- Increasing depth
- Using uniform 3×3 filters
- Better feature extraction

---

9) What is Skip-Gram? (With Code)

Skip-Gram is the second architecture of Word2Vec.

It predicts surrounding context words from a given target word.

Example:

Sentence: "The cat sits on the mat"

Target word = "sits"
Output = ["cat", "on"]

Advantages:

- Works well with small datasets
- Better for rare words
- Slightly slower than CBOW

Code (Gensim)

```
from gensim.models import Word2Vec

sentences = [
    ["the", "cat", "sits", "on", "the", "mat"],
    ["the", "dog", "runs", "in", "the", "park"]
]

model = Word2Vec(sentences, vector_size=100, window=2, min_count=1, sg=1)

print(model.wv["dog"])
```

sg=1 indicates Skip-Gram.